

Co-1 _Assessment_Tool_2 - Flowgorithm

1. Read the coordinates (x, y) of a point and identify the quadrant or axis on which the point lies.

Answer:

Function Main

Declare Real x

Declare Real y

Input x

Input y

If $x > 0$ and $y > 0$

Output "Quadrant I"

Else

If $x < 0$ and $y > 0$

Output "Quadrant II"

Else

If $x < 0$ and $y < 0$

Output "Quadrant III"

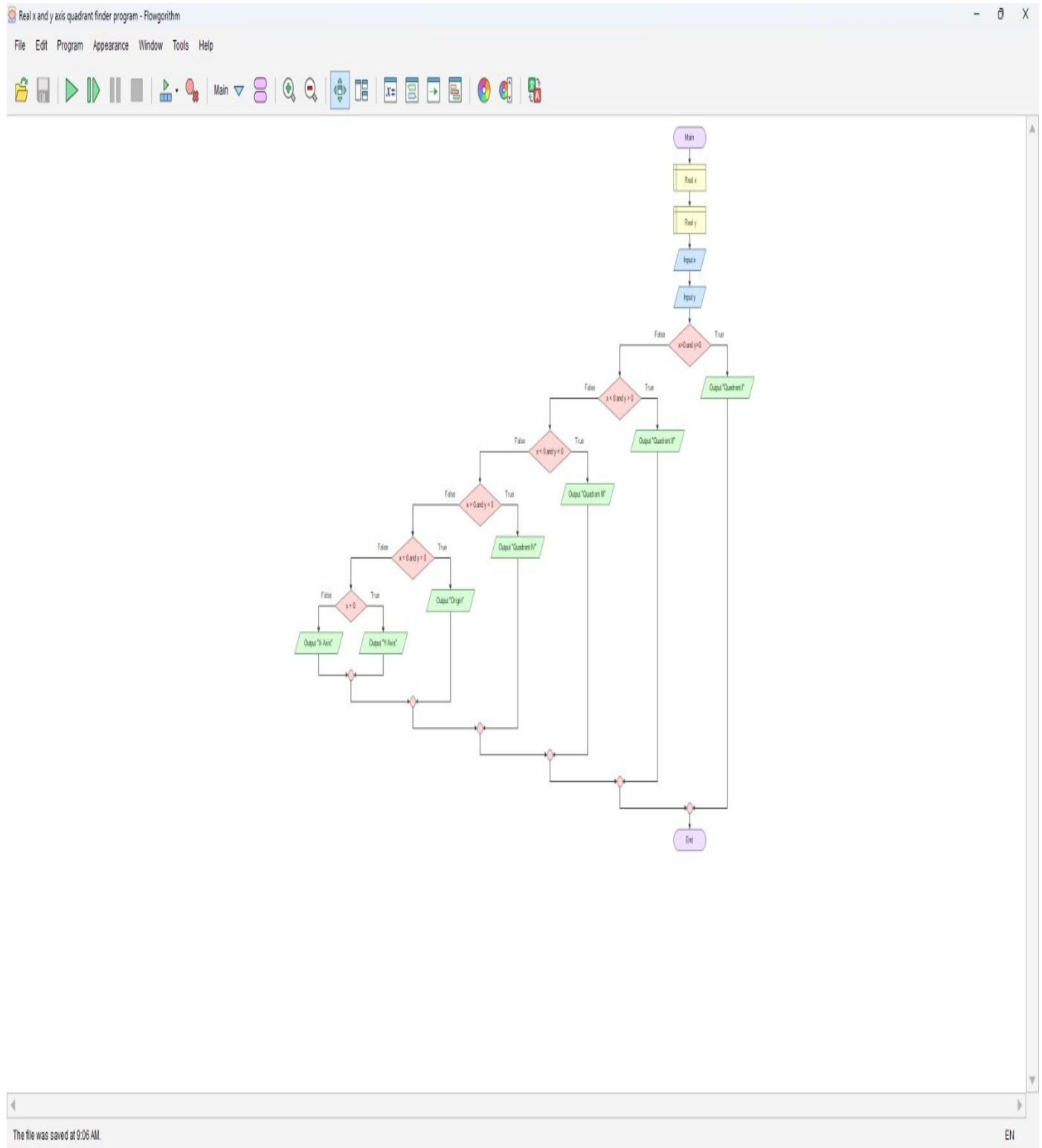
Else

If $x > 0$ and $y < 0$

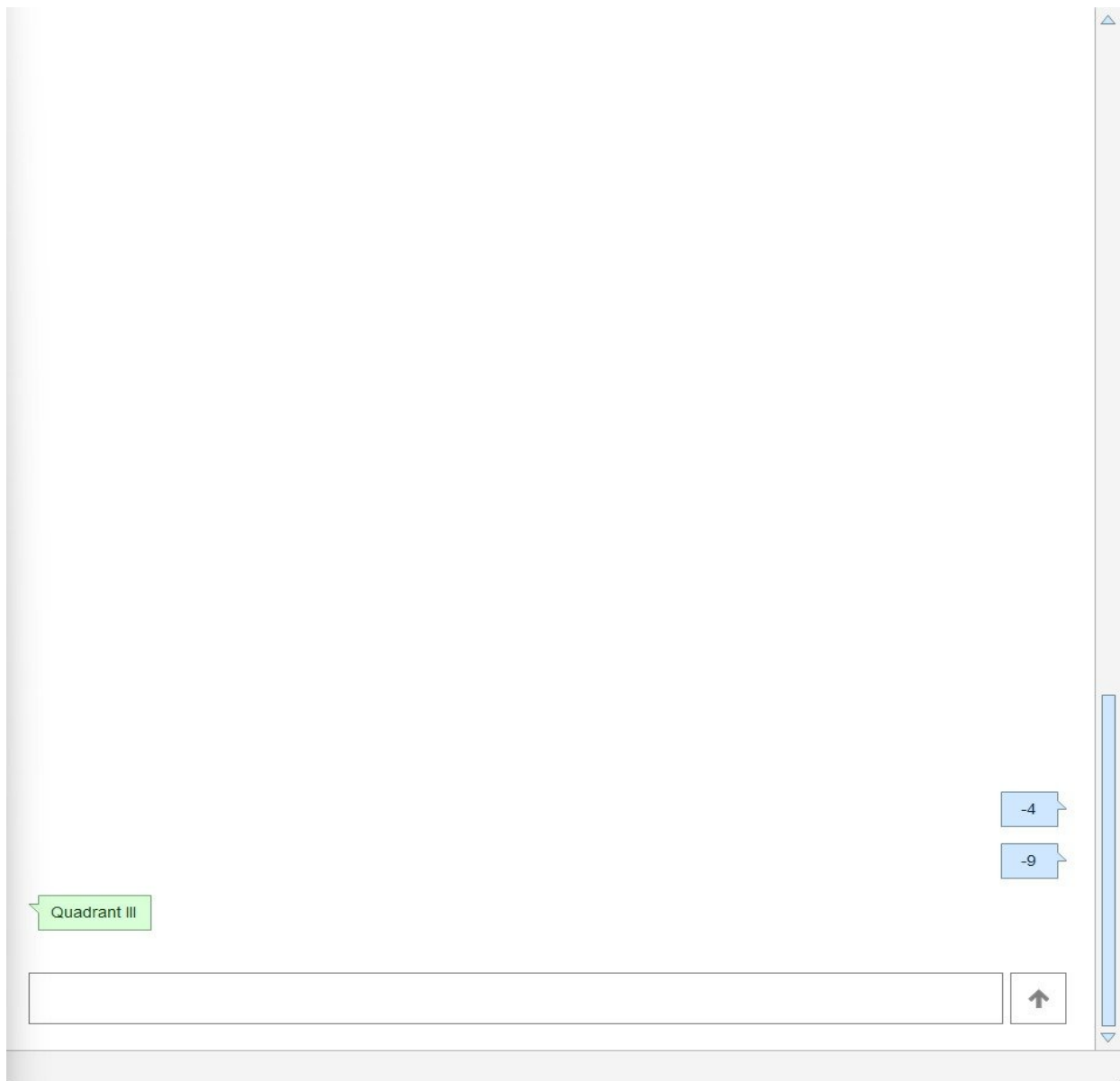
Output "Quadrant IV"

```
    Else
      If x = 0 and y = 0
        Output "Origin"
      Else
        If x = 0
          Output "Y-Axis"
        Else
          Output "X-Axis"
        End
      End
    End
  End
End
End
End
```

FLOWCHART:



OUTPUT:



2. Read two positive integers and calculate their GCD using repeated division; then compute the LCM.

Answer:

Function Main

Declare Integer a

Declare Integer b

Declare Integer temp1

Declare Integer temp2

Declare Integer gcd

Declare Integer lcm

Input a

Input b

Assign temp1 = a

Assign temp2 = b

While temp2 $\neq$ 0

Assign gcd = temp2

Assign temp2 = temp1 Mod temp2

Assign temp1 = gcd

End

Assign gcd = temp1

Assign lcm = (a * b) / gcd

Output "GCD = "

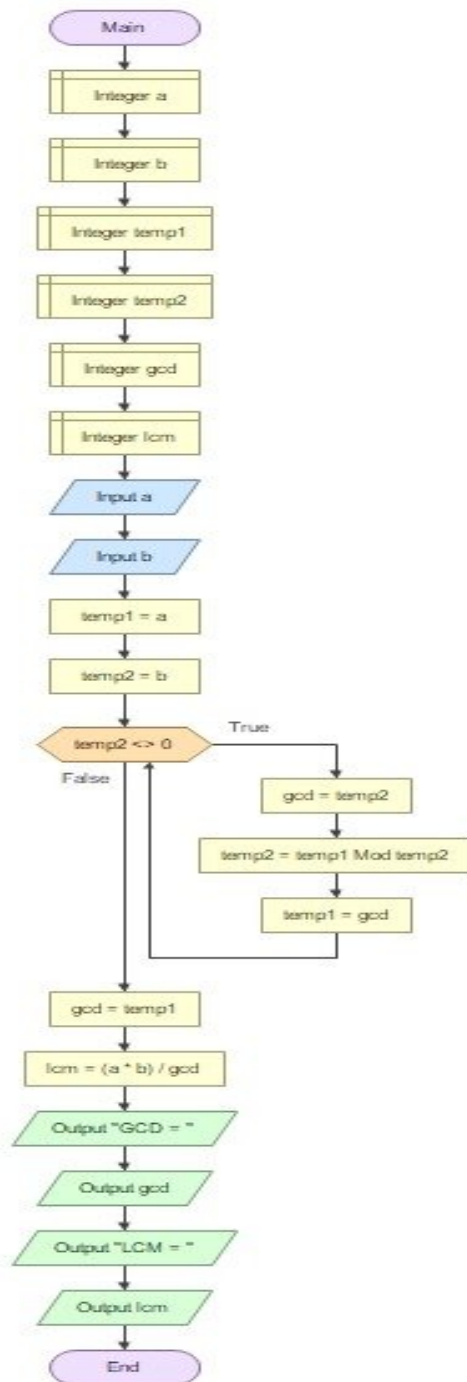
Output gcd

Output "LCM = "

Output lcm

End

FLOWCHART:



OUTPUT:

GCD =

6

LCM =

36

12

18



3.s

3. Design an authentication flowchart with functions to validate username, password length, and character categories. Combine their Boolean results for login approval.

Answer:

Function Main

Declare String username

Declare String password

Declare Boolean usernameOK

Declare Boolean lengthOK

Declare Boolean categoryOK

Declare Boolean loginApproved

Input username

Input password

Assign usernameOK = validateUsername(username)

Assign lengthOK = validatePasswordLength(password)

Assign categoryOK = validateCharacterCategories(password)

Assign loginApproved = usernameOK and lengthOK and categoryOK

If loginApproved = True

Output "Login Approved"

Else

Output "Login Denied"

End

End

Function validateUsername Returns Boolean

Parameters String username

Declare Boolean isValid

Assign isValid = False

If Length(username) >= 5 and Not Contains(username, " ")

Assign isValid = True

End

Return isValid

End

Function validatePasswordLength Returns Boolean

Parameters String password

Declare Boolean isValid

Assign isValid = False

If Length(password) >= 8

Assign isValid = True

End

Return isValid

End

Function validateCharacterCategories Returns Boolean

Parameters String password

Declare Boolean hasUpper

Declare Boolean hasLower

Declare Boolean hasDigit

Declare Boolean hasSpecial

Declare Integer i

Declare String c

Assign hasUpper = False

Assign hasLower = False

Assign hasDigit = False

Assign hasSpecial = False

Assign i = 1

While i <= Length(password)

Assign c = Mid(password, i, 1)

If IsUpper(c)

Assign hasUpper = True

End

If IsLower(c)

Assign hasLower = True

End

If IsDigit(c)

Assign hasDigit = True

End

If Not IsUpper(c) and Not IsLower(c) and Not IsDigit(c)

Assign hasSpecial = True

End

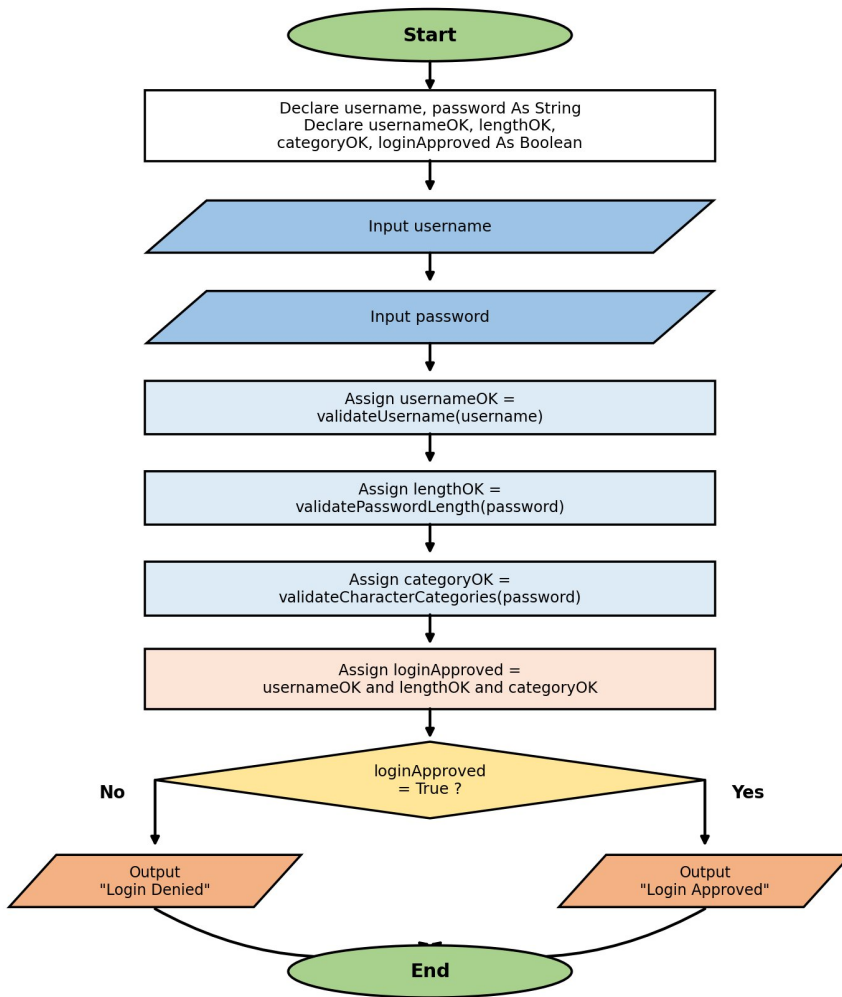
Assign i = i + 1

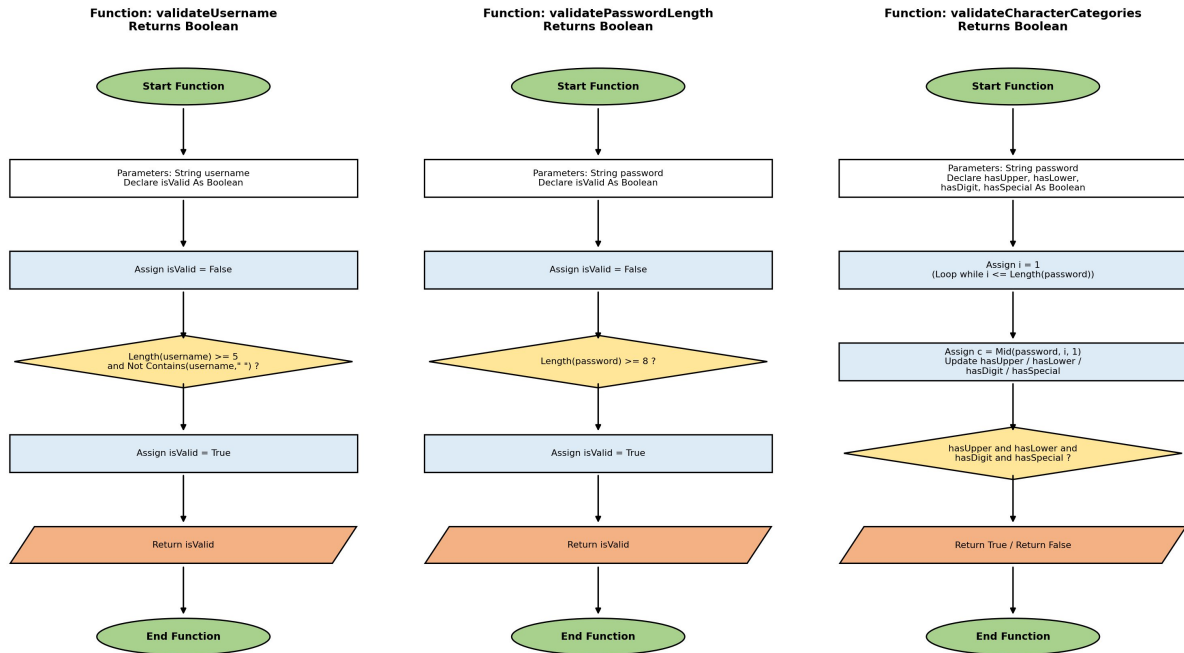
End

Return hasUpper and hasLower and hasDigit and hasSpecial

End

FLOWCHART:





OUTPUT:

Output

```

Enter username: j.smith
Enter password: Passw0rd!
Login Approved

Enter username: al
Enter password: Passw0rd!
Login Denied

```